

# REPORTE TAREA 2

## ALGORITMOS Y COMPLEJIDAD

*«¿Quien ganará? Greedys vs Dinámico vs Bruto.»*

Rodrigo Cáceres

22 de junio de 2026

23:55

### 1. Entorno experimental

**La extensión máxima para esta sección es de media página.**

Los experimentos fueron realizados en un computador con sistema operativo [SISTEMA OPERATIVO], procesador [PROCESADOR], memoria RAM [RAM] y utilizando el compilador [COMPILADOR] en su versión [VERSIÓN]. El desarrollo y la ejecución de los programas se hicieron en [ENTORNO / EDITOR / SHELL, si corresponde].

Para las mediciones se utilizaron las familias de casos generadas automáticamente para la tarea, las cuales siguen el formato descrito en el enunciado. En particular, se consideraron instancias pequeñas para validar correctitud y familias controladas para estudiar el comportamiento de los algoritmos al variar la cantidad de animes, la cantidad de capítulos, los minutos disponibles y la energía disponible.

Los archivos de entrada, salida y medición fueron generados por los scripts del proyecto, de modo que los resultados presentados puedan reproducirse a partir del código fuente y de los casos de prueba correspondientes.

### 2. Resultados y Análisis

**La extensión máxima para esta sección es de 4 páginas.** Si utiliza menos páginas y explica bien sus resultados, es preferible.

En esta sección se presentan los resultados experimentales obtenidos al ejecutar las cuatro implementaciones sobre las familias de casos generadas automáticamente. Los casos pequeños (small) se utilizaron para validar correctitud y comparar fuerza bruta con programación dinámica; las familias vary\_n, vary\_q, vary\_m y vary\_e se construyeron para aislar, respectivamente, la cantidad de animes, la cantidad total de capítulos, los minutos disponibles y la energía disponible.

## 2.1. Síntesis global

La Tabla 1 resume el comportamiento promedio observado en las ejecuciones realizadas. La programación dinámica es claramente más costosa en tiempo y memoria que los métodos greedy, pero es la única implementación exacta; por ello se usa como referencia para evaluar la calidad de las heurísticas.

| Algoritmo  | Tiempo promedio (ms)    | Memoria promedio (KB) | height | Fuerza bruta |
|------------|-------------------------|-----------------------|--------|--------------|
| —          | — Programación dinámica |                       |        | —            |
| — Greedy 1 | —                       |                       |        | — Greedy 2   |
| —          | — height                |                       |        |              |

Cuadro 1: Resumen global de tiempo y memoria. Los valores deben leerse como promedios sobre los casos efectivamente ejecutados por cada algoritmo.

## 2.2. Tiempo de ejecución

La Figura 1 muestra el tiempo promedio según la cantidad de animes. Se observa que la programación dinámica crece de manera mucho más pronunciada que los métodos greedy, mientras que estos últimos se mantienen cercanos a una línea casi plana. Esto concuerda con la diferencia esperada entre un algoritmo exacto que explora un espacio de estados más grande y heurísticas que toman decisiones locales.

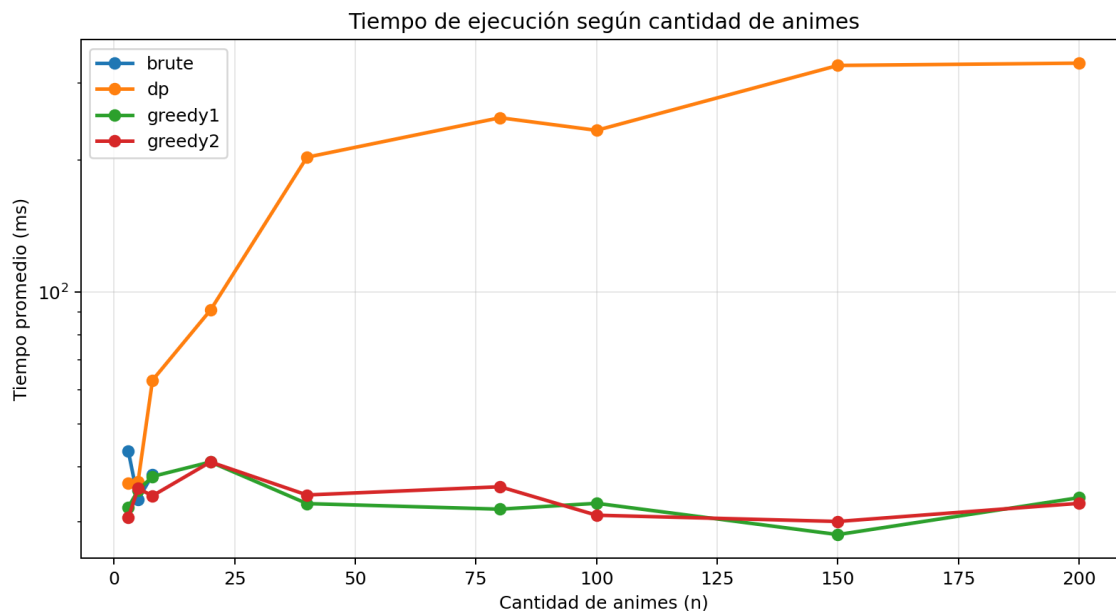


Figura 1: Tiempo de ejecución según cantidad de animes. La escala vertical está en logaritmo para distinguir mejor la separación entre los métodos.

La Figura 2 presenta el tiempo según la cantidad total de capítulos. Aquí la dispersión es mayor porque esta variable no se aisló por completo: cada caso pertenece a una familia distinta y, además de variar

el número de capítulos, también pueden cambiar el número de animes, los presupuestos disponibles y la distribución interna de las duraciones y energías. Aun así, se mantiene la tendencia general: programación dinámica es el método más costoso, mientras que los greedy y la fuerza bruta en casos pequeños son mucho más livianos.

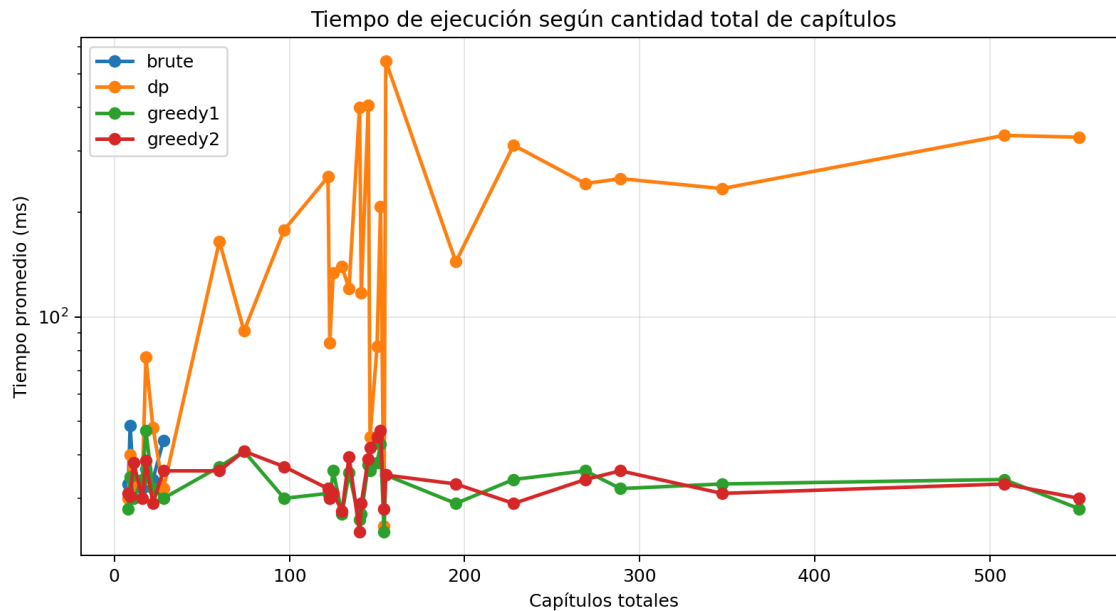


Figura 2: Tiempo de ejecución según cantidad total de capítulos.

Las Figuras 3 y 4 muestran el comportamiento respecto de los presupuestos de minutos y energía, respectivamente. En ambos casos la programación dinámica sigue siendo la que más se resiente cuando el espacio de estados crece, mientras que los greedy apenas cambian. Las pequeñas oscilaciones en las curvas no son una anomalía: provienen de que cada punto corresponde a una instancia distinta y, por tanto, el costo real depende también de la estructura interna de los animes y no solo del valor del presupuesto.

### 2.3. Uso de memoria

La memoria presenta un patrón más estable que el tiempo, aunque también exhibe variaciones entre familias. En los gráficos, los métodos greedy se mantienen cerca de un valor base muy similar entre sí, lo que es esperable porque su lógica solo requiere estructuras pequeñas para ordenar y evaluar opciones. En contraste, programación dinámica utiliza tablas de estados y, por ello, su consumo de memoria es mayor.

La Figura 5 resume el uso de memoria según la cantidad de animes. Se aprecia que los greedy prácticamente permanecen en un nivel de referencia, mientras que programación dinámica incrementa su consumo a medida que crece el problema. Esto es coherente con el hecho de que DP necesita almacenar información adicional para combinar subproblemas.

Las Figuras 6, 7 y 8 muestran que la memoria no varía de forma estrictamente monótona. Esto no

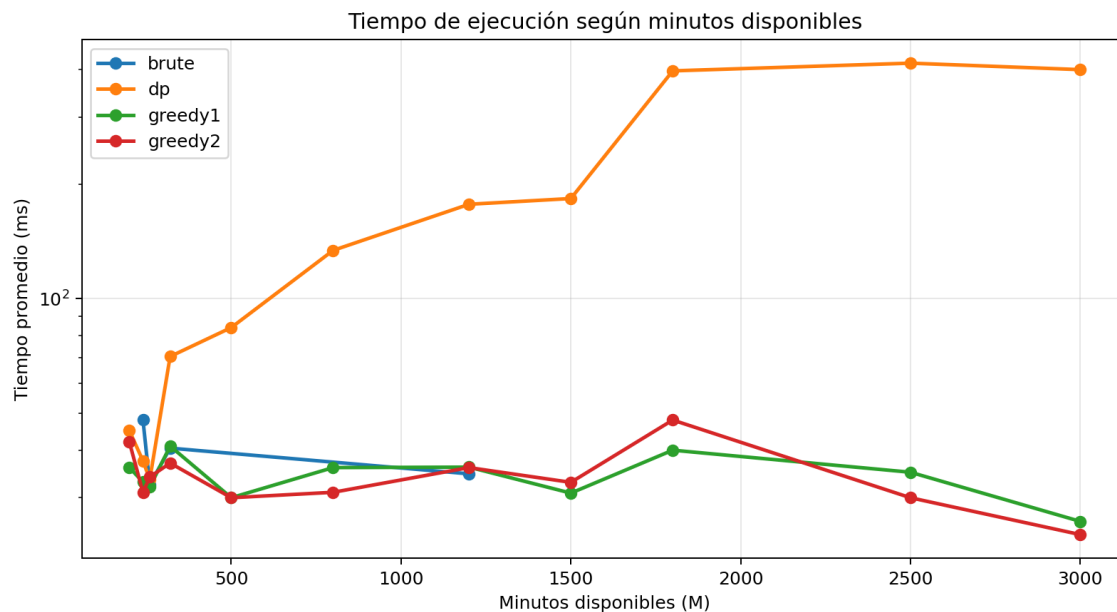


Figura 3: Tiempo de ejecución según minutos disponibles.

necesariamente indica un error: la medición corresponde al *peak working set* reportado por el sistema operativo, por lo que puede verse afectada por el runtime, la reserva interna de memoria y la distribución concreta de cada instancia. En consecuencia, las curvas deben interpretarse como una tendencia general y no como una función exacta del parámetro mostrado en el eje horizontal.

## 2.4. Comparación de calidad respecto del óptimo

La Figura 9 compara la solución de los métodos greedy con el óptimo entregado por programación dinámica, usando la razón  $\left[ \text{calidad} = \frac{\text{valor greedy}}{\text{valor óptimo}} \right]$ . Como la referencia óptima proviene de DP, el valor 1 representa igualdad con el óptimo.

## 2.5. Comentarios finales sobre las mediciones

La fuerza bruta solo se ejecutó en instancias pequeñas, porque su costo crece muy rápido y no es razonable exigirle los casos medianos y grandes. En cambio, programación dinámica entrega la referencia exacta para comparar correctitud y calidad, mientras que los métodos greedy permiten estudiar el trade-off entre rapidez y precisión. En conjunto, las mediciones muestran el comportamiento esperado: exactitud a mayor costo para DP, y rapidez con menor calidad para las heurísticas.

## 3. Conclusiones

**La extensión máxima para esta sección es un párrafo breve (máximo 1/4 de página).**

Escriba una conclusión breve que interprete los resultados más importantes. Cada afirmación debe estar respaldada por los datos presentados.

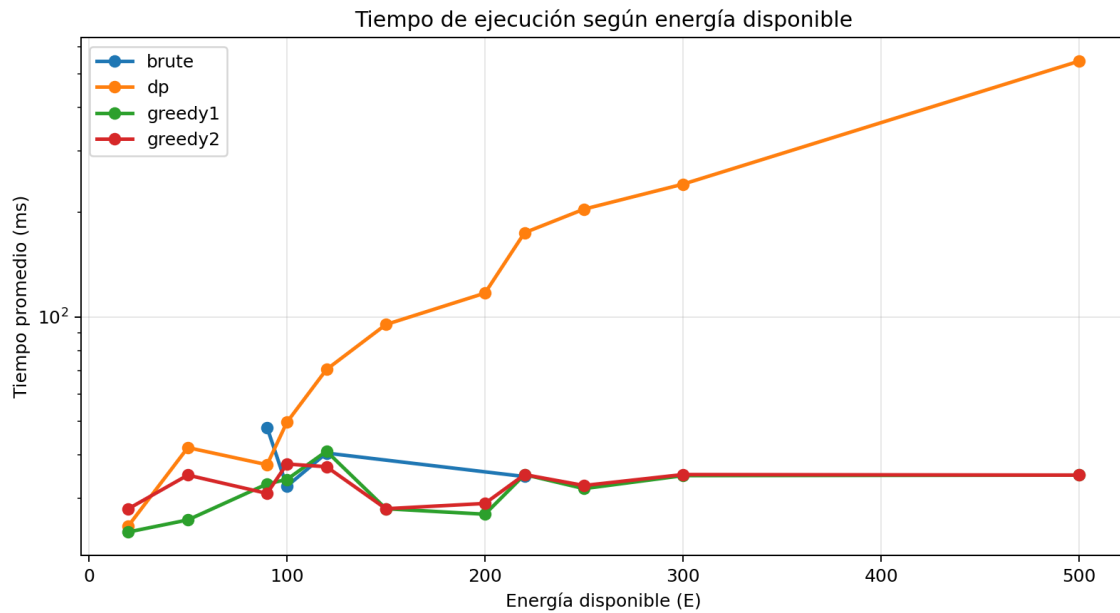


Figura 4: Tiempo de ejecución según energía disponible.

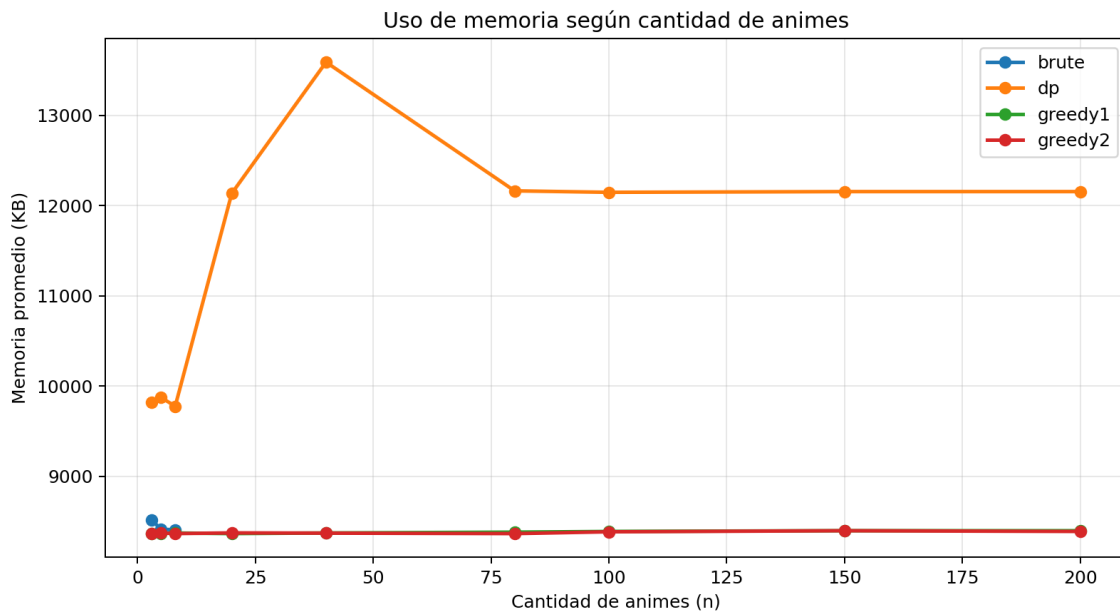


Figura 5: Uso de memoria según cantidad de animes.

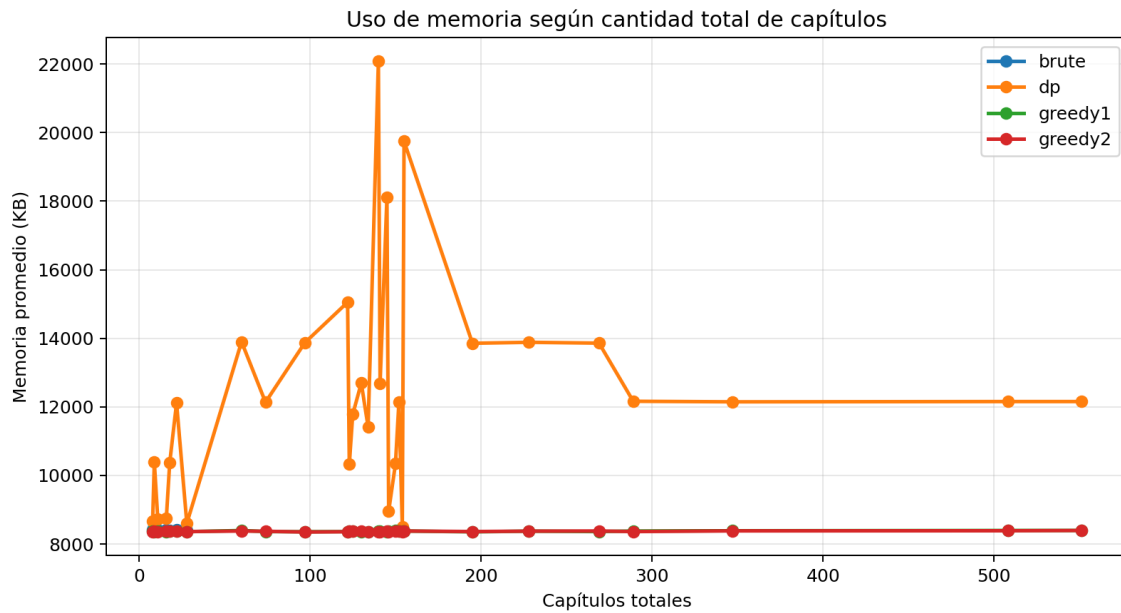


Figura 6: Uso de memoria según cantidad total de capítulos.

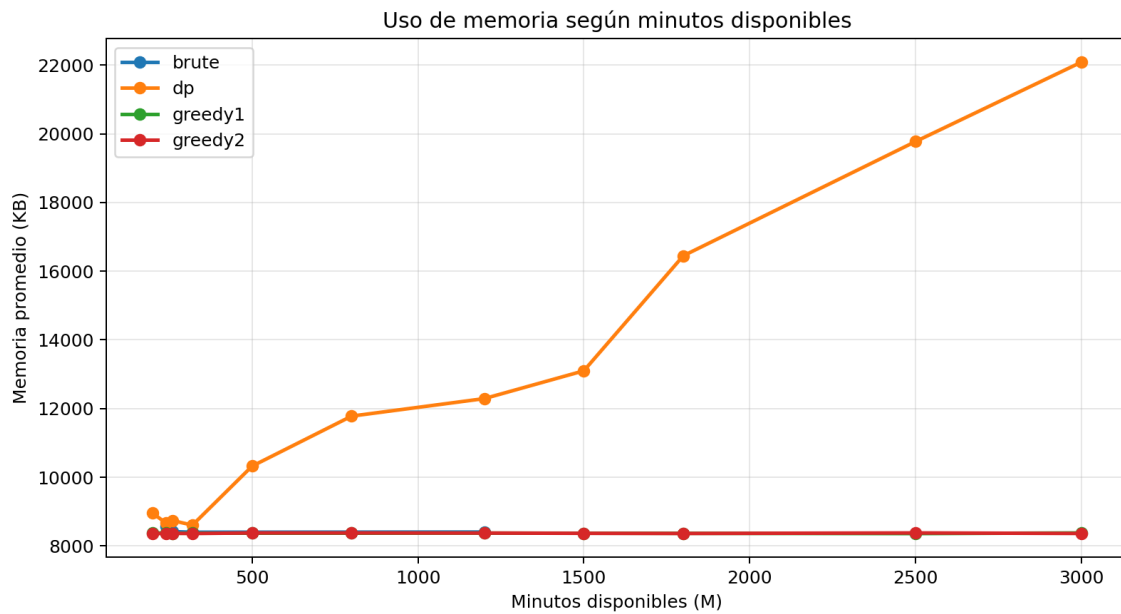


Figura 7: Uso de memoria según minutos disponibles.

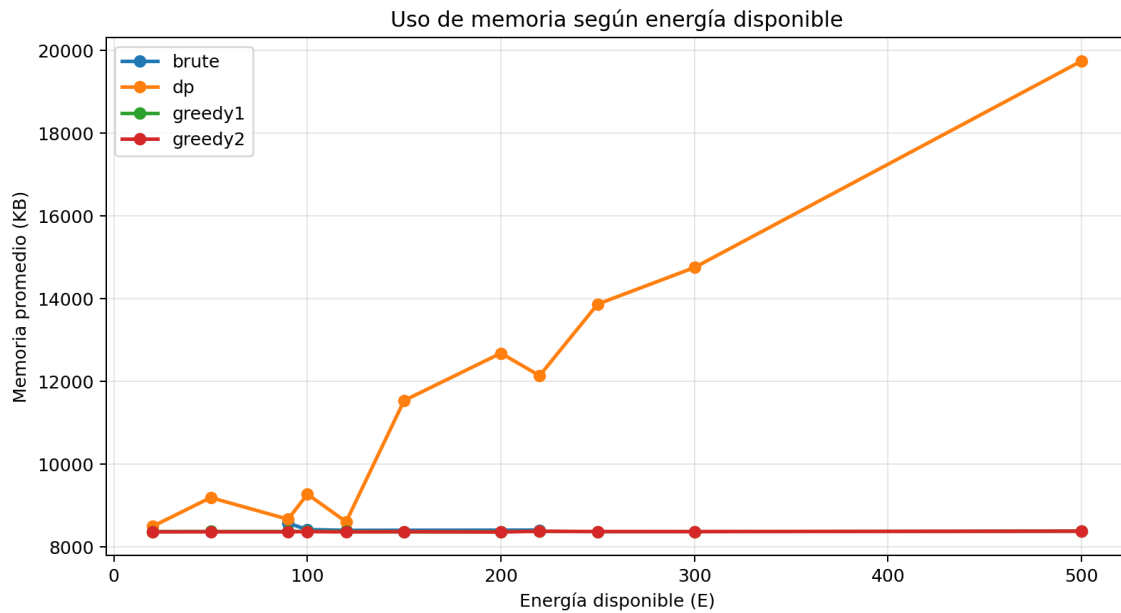


Figura 8: Uso de memoria según energía disponible.

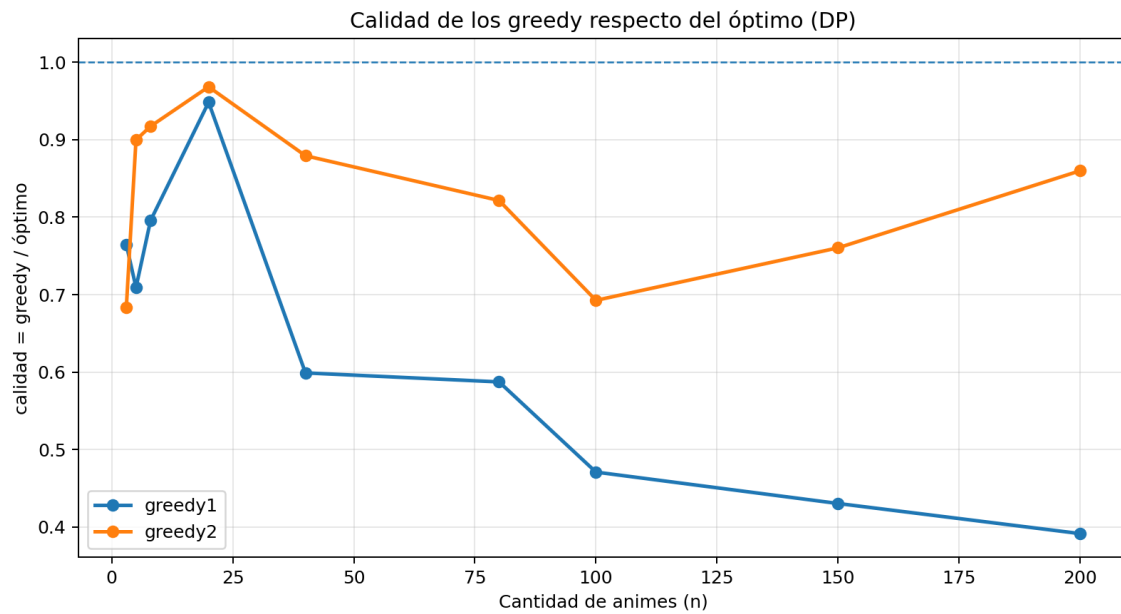


Figura 9: Calidad de solución de los greedy respecto del óptimo entregado por programación dinámica.